🤖 MACHINE LEARNING MASTERY QUIZ

Comprehensive Assessment — 100 Questions

High-Level Theory | Advanced Coding & Implementation

Total Questions	**100**	**Marks / Question**	**1 Mark**
Total Marks	**100**	**Time Allowed**	**120 Minutes**

📋 INSTRUCTIONS

- Each question carries 1 mark. There is NO negative marking.
- Choose only ONE answer per question. Write A / B / C / D in the 'Ans:' box next to each question.
- Section 1 (Q1–50): High-Level Theory • Section 2 (Q51–100): Advanced Coding & Implementation
- **Complete the Student Details page FIRST before turning to the questions.**

✏️ STUDENT DETAILS

Fill in all fields completely before attempting the quiz

👤 **Student Name**	
🎓 **USN / Roll No**	
🏛️ **Branch / Dept**	
📧 **Email ID**	

⚠️ **Do NOT proceed to the quiz until ALL Student Details above are filled in**

SECTION 1 — HIGH-LEVEL THEORY

Questions 1 – 50 | Concepts, Algorithms, Evaluation & Advanced Topics

Q1.	**Which of the following best defines the Bias-Variance Tradeoff in Machine Learning?**
A)	High bias leads to overfitting; high variance leads to underfitting
B)	High bias leads to underfitting; high variance leads to overfitting; the tradeoff seeks to minimise total error
C)	Bias and variance are always inversely proportional in all models
D)	Bias refers to data errors; variance refers to model complexity
Ans:	

Q2.	**What is the primary purpose of cross-validation in machine learning?**
A)	To increase training data size
B)	To estimate model generalisation performance on unseen data by rotating the train/test split
C)	To reduce the number of features
D)	To replace the need for a test set entirely
Ans:	

Q3.	**In supervised learning, what is the difference between regression and classification?**
A)	Regression predicts discrete class labels; classification predicts continuous values
B)	Regression predicts continuous output values; classification predicts discrete class labels
C)	Both predict the same type of output
D)	Regression is unsupervised; classification is supervised
Ans:	

Q4.	**What does the No Free Lunch theorem state about machine learning algorithms?**
A)	Deep learning always outperforms classical methods
B)	No single learning algorithm is universally superior across all possible problems
C)	More data always leads to better models
D)	Gradient descent always converges to the global optimum
Ans:	

Q5.	**Which metric is most appropriate for evaluating a binary classifier on a highly imbalanced dataset?**

A)	Accuracy
B)	Mean Squared Error
C)	F1-Score or AUC-ROC
D)	R-squared
Ans:	

Q6.	What is the kernel trick in Support Vector Machines?
A)	A method to prune decision trees
B)	Implicitly mapping data into a higher-dimensional feature space using a kernel function without computing the transformation explicitly
C)	A regularisation technique for neural networks
D)	A way to reduce the learning rate during training
Ans:	

Q7.	In a Decision Tree, what does information gain measure?
A)	The total number of leaf nodes
B)	The reduction in entropy (or impurity) achieved by splitting on a particular feature
C)	The depth of the tree after pruning
D)	The accuracy on the training set
Ans:	

Q8.	Which of the following correctly describes the Random Forest algorithm?
A)	A single deep decision tree trained on all features
B)	An ensemble of decision trees trained on random subsets of data and features, aggregating predictions by voting or averaging
C)	A boosting algorithm that sequentially corrects errors
D)	A neural network with randomly initialised weights
Ans:	

Q9.	What is the key difference between Bagging and Boosting ensemble methods?
A)	Bagging trains models sequentially, each correcting the previous; Boosting trains models in parallel on random subsets
B)	Bagging trains models in parallel on random subsets to reduce variance; Boosting trains sequentially to reduce bias
C)	They are identical techniques
D)	Bagging is only used for classification; Boosting only for regression

Ans:	

Q10.	**What does the k in k-Nearest Neighbours (kNN) control?**
A)	The number of features used
B)	The number of nearest training examples consulted when making a prediction; smaller k increases variance
C)	The number of clusters formed
D)	The learning rate of the algorithm
Ans:	

Q11.	**In k-Means clustering, what is the objective function being minimised?**
A)	Cross-entropy loss
B)	Within-cluster sum of squared distances from each point to its centroid (inertia)
C)	Log-likelihood of the data
D)	Silhouette coefficient
Ans:	

Q12.	**What is the main disadvantage of k-Means clustering?**
A)	It cannot handle numeric features
B)	It requires the number of clusters k to be specified in advance and is sensitive to initialisation and outliers
C)	It always produces unequal cluster sizes
D)	It does not use distance metrics
Ans:	

Q13.	**What does PCA (Principal Component Analysis) aim to achieve?**
A)	Classify data into predefined categories
B)	Find orthogonal directions of maximum variance to project data into a lower-dimensional space
C)	Cluster data without labels
D)	Regularise a regression model
Ans:	

Q14.	**How does DBSCAN differ from k-Means clustering?**
A)	DBSCAN requires k clusters to be specified; k-Means does not
B)	DBSCAN discovers clusters of arbitrary shape based on density and can label outliers as noise; k-Means assumes spherical clusters

C)	DBSCAN is a supervised algorithm
D)	DBSCAN cannot handle high-dimensional data
Ans:	

Q15.	What is the Silhouette Score used for?
A)	Measuring model training loss
B)	Evaluating cluster quality by comparing intra-cluster cohesion with inter-cluster separation; values closer to 1 indicate well-separated clusters
C)	Computing feature importance
D)	Selecting the learning rate
Ans:	

Q16.	What problem does the Vanishing Gradient problem cause in deep neural networks?
A)	The loss becomes infinite during training
B)	Gradients shrink exponentially as they backpropagate through layers, causing early layers to learn very slowly or not at all
C)	The model converges too quickly
D)	Weights become too large
Ans:	

Q17.	What is the purpose of Batch Normalisation in deep learning?
A)	To increase the batch size automatically
B)	To normalise layer inputs across the batch, stabilising and accelerating training by reducing internal covariate shift
C)	To prevent overfitting by randomly dropping neurons
D)	To initialise weights close to zero
Ans:	

Q18.	How does Dropout regularisation work in neural networks?
A)	It removes low-importance features from input
B)	During training, it randomly sets a fraction of neuron activations to zero, preventing co-adaptation and reducing overfitting
C)	It reduces the learning rate at each epoch
D)	It adds L2 penalty to all weights
Ans:	

Q19.	What is the role of an activation function in a neural network?
A)	To compute the weighted sum of inputs
B)	To introduce non-linearity, enabling the network to learn complex, non-linear mappings
C)	To initialise network weights
D)	To normalise the output layer
Ans:	

Q20.	Which activation function is most commonly used in hidden layers of modern deep networks and why?
A)	Sigmoid, because it outputs probabilities
B)	Tanh, because it is zero-centred
C)	ReLU (Rectified Linear Unit), because it mitigates vanishing gradients and is computationally efficient
D)	Softmax, because it handles multi-class output
Ans:	

Q21.	What is the difference between Stochastic Gradient Descent (SGD) and Mini-Batch Gradient Descent?
A)	SGD uses the full dataset per update; mini-batch uses one sample
B)	SGD updates weights using one sample per step; mini-batch uses a small random subset, balancing noise and efficiency
C)	They are identical algorithms
D)	Mini-batch is always slower than SGD
Ans:	

Q22.	What does the Adam optimiser combine?
A)	L1 and L2 regularisation
B)	Momentum (first moment) and RMSProp (second moment) to adapt the learning rate per parameter
C)	Dropout and batch normalisation
D)	Gradient clipping and weight decay
Ans:	

Q23.	What is learning rate scheduling and why is it used?
A)	Randomly changing the model architecture during training
B)	Systematically reducing the learning rate over time to allow fine-grained convergence after initial rapid progress
C)	Increasing the learning rate to escape local minima

D)	Fixing the learning rate at a high value throughout training
Ans:	

Q24.	What is the purpose of L1 regularisation (Lasso) versus L2 regularisation (Ridge)?
A)	Both produce sparse weight vectors
B)	L1 promotes sparsity by driving some weights to exactly zero (feature selection); L2 shrinks all weights toward zero without eliminating them
C)	L2 promotes sparsity; L1 prevents overfitting only
D)	They are equivalent techniques
Ans:	

Q25.	What is early stopping in the context of model training?
A)	Stopping training when training loss reaches zero
B)	Halting training when validation performance stops improving, to prevent overfitting
C)	Removing neurons that do not contribute to the output
D)	Reducing batch size partway through training
Ans:	

Q26.	What is feature scaling and why is it important for algorithms like SVM and kNN?
A)	Converting categorical features to numbers; important for tree-based models
B)	Standardising or normalising feature ranges so that no single feature dominates distance or gradient calculations
C)	Removing correlated features from the dataset
D)	Adding polynomial features to improve model capacity
Ans:	

Q27.	What does one-hot encoding achieve for categorical variables?
A)	Assigns ordinal integer labels to categories
B)	Creates a binary indicator column for each category, avoiding imposing false ordinal relationships between categories
C)	Reduces the number of features
D)	Converts text to numerical embeddings
Ans:	

Q28.	What is the curse of dimensionality?
A)	Too many training samples making computation slow

B)	The phenomenon where data becomes increasingly sparse as the number of dimensions grows, degrading distance-based algorithms
C)	The inability of deep networks to handle many features
D)	Overfitting caused by too many hyperparameters
Ans:	

Q29.	**What is mutual information used for in feature selection?**
A)	Measuring model accuracy on a test set
B)	Quantifying the statistical dependence between a feature and the target variable, including non-linear relationships
C)	Computing pairwise feature correlation using Pearson's coefficient
D)	Selecting features with the highest variance
Ans:	

Q30.	**What is the difference between filter, wrapper, and embedded feature selection methods?**
A)	All three use cross-validation to select features
B)	Filter methods use statistical scores independently of the model; wrapper methods evaluate feature subsets using a model; embedded methods learn feature importance during training
C)	Wrapper methods are the fastest
D)	Embedded methods do not depend on the learning algorithm
Ans:	

Q31.	**What does the ROC curve plot and what does AUC represent?**
A)	Precision vs Recall; AUC represents model accuracy
B)	True Positive Rate (Recall) vs False Positive Rate at various thresholds; AUC represents the probability that the model ranks a random positive above a random negative
C)	F1-Score vs threshold; AUC is the average F1
D)	Training loss vs validation loss over epochs
Ans:	

Q32.	**In a confusion matrix, what does a high number of False Negatives indicate?**
A)	The model predicts too many positives
B)	The model frequently misses actual positive cases (low recall), which is critical in medical diagnosis contexts
C)	The model has high precision
D)	The model is well-calibrated

Ans:	

Q33.	**What is the difference between precision and recall?**
A)	Precision measures true negatives; recall measures true positives
B)	Precision measures the fraction of predicted positives that are actually positive; recall measures the fraction of actual positives correctly identified
C)	They are always equal for balanced datasets
D)	Precision is used for regression; recall for classification
Ans:	

Q34.	**What is Mean Absolute Error (MAE) and how does it differ from RMSE?**
A)	MAE squares errors making it sensitive to outliers; RMSE does not
B)	MAE computes the average absolute difference between predictions and targets; RMSE squares errors before averaging, penalising large errors more heavily
C)	They are identical metrics
D)	RMSE is used for classification; MAE for regression
Ans:	

Q35.	**What is the purpose of a learning curve (training size vs. error) in ML diagnostics?**
A)	To tune hyperparameters automatically
B)	To diagnose whether a model suffers from high bias (underfitting) or high variance (overfitting) based on how train/validation error evolves with data size
C)	To visualise feature importance
D)	To determine the optimal batch size
Ans:	

Q36.	**What is Transfer Learning and what is its primary advantage?**
A)	Training a model from scratch on a large dataset
B)	Reusing a model trained on a source task as a starting point for a target task, leveraging learned representations and requiring less data/compute
C)	Transferring data from one domain to another
D)	Using multiple models on the same task simultaneously
Ans:	

Q37.	**What is the difference between generative and discriminative models?**
A)	Generative models predict class labels; discriminative models generate new samples

B)	Generative models learn the joint distribution P(X, Y) and can generate new data; discriminative models learn P(Y|X) directly for classification
C)	They are mathematically equivalent
D)	Discriminative models are always more accurate
Ans:	

Q38.	What is an attention mechanism in deep learning?
A)	A technique to randomly focus on features during training
B)	A mechanism that computes a weighted sum of input representations, allowing the model to dynamically focus on the most relevant parts of the input for each output
C)	A type of pooling layer in CNNs
D)	A regularisation method for RNNs
Ans:	

Q39.	What is the vanishing gradient problem in RNNs and how do LSTMs address it?
A)	LSTMs use larger batch sizes to stabilise gradients
B)	In RNNs gradients shrink through many timesteps; LSTMs introduce gating mechanisms (input, forget, output gates) that control gradient flow, enabling learning of long-range dependencies
C)	LSTMs use skip connections like ResNets
D)	LSTMs replace recurrence with self-attention
Ans:	

Q40.	What is the purpose of word embeddings (e.g., Word2Vec, GloVe) in NLP?
A)	To one-hot encode words in a vocabulary
B)	To represent words as dense low-dimensional vectors capturing semantic and syntactic relationships, enabling models to generalise across similar words
C)	To remove stop words from text
D)	To convert sentences to fixed-length binary codes
Ans:	

Q41.	What is Reinforcement Learning and how does it differ from supervised learning?
A)	RL uses labelled data; supervised learning uses rewards
B)	RL trains an agent to maximise cumulative reward through trial-and-error interaction with an environment; supervised learning learns from labelled input-output pairs
C)	Both use the same loss function
D)	RL does not require a model or environment
Ans:	

Q42.	What is the Expectation-Maximisation (EM) algorithm used for in ML?
A)	Training deep neural networks
B)	Finding maximum likelihood estimates for models with latent (hidden) variables by alternating between estimating latent variables (E-step) and maximising parameters (M-step)
C)	Performing gradient descent on convex functions
D)	Selecting features using mutual information
Ans:	

Q43.	What is the difference between parametric and non-parametric models?
A)	Parametric models have infinite parameters; non-parametric have fixed parameters
B)	Parametric models assume a fixed functional form with a finite number of parameters; non-parametric models grow in complexity with data (e.g., kNN, kernel methods)
C)	Non-parametric models cannot generalise to new data
D)	Parametric models are always faster to train
Ans:	

Q44.	What is Gradient Boosting and how does XGBoost improve upon it?
A)	Gradient Boosting trains trees in parallel; XGBoost trains sequentially
B)	Gradient Boosting sequentially adds trees to correct residual errors; XGBoost adds regularisation, second-order gradients, column subsampling, and hardware optimisations for speed and accuracy
C)	XGBoost replaces trees with neural networks
D)	They are identical algorithms
Ans:	

Q45.	What is the purpose of hyperparameter tuning and what is the difference between Grid Search and Random Search?
A)	Grid Search samples random hyperparameters; Random Search tries all combinations
B)	Hyperparameter tuning finds optimal model configuration; Grid Search exhaustively tries all combinations; Random Search samples randomly, often finding good solutions faster in high-dimensional spaces
C)	Both methods guarantee finding the global optimum
D)	Hyperparameter tuning is only needed for deep learning
Ans:	

Q46.	What is a Gaussian Process and what is its key property?
A)	A type of deep neural network with Gaussian activations

B)	A probabilistic model that defines a distribution over functions, providing not just predictions but also principled uncertainty estimates at each point
C)	A clustering algorithm based on Gaussian mixture models
D)	An optimisation algorithm for convex functions
Ans:	

Q47.	What is the difference between online learning and batch learning?
A)	Online learning uses the full dataset; batch uses mini-batches
B)	Online learning updates the model incrementally as each new data point arrives; batch learning trains on the entire dataset before updating
C)	They produce identical models
D)	Online learning is only applicable to neural networks
Ans:	

Q48.	What is federated learning and why is it used?
A)	Training a model on a central server with all data aggregated
B)	Training models locally on distributed devices and aggregating only model updates (not raw data) to preserve privacy
C)	A technique to combine multiple datasets from different sources
D)	A distributed computing framework for big data processing
Ans:	

Q49.	What does the term 'model calibration' mean in ML?
A)	Tuning the number of layers in a neural network
B)	Ensuring that predicted probabilities align with actual frequencies — a model predicting 80% probability should be correct 80% of the time
C)	Normalising input features
D)	Evaluating model accuracy on a holdout set
Ans:	

Q50.	What is the Shapley value used for in interpretable ML (e.g., SHAP)?
A)	Computing the gradient of the loss with respect to inputs
B)	A game-theory-based method that fairly attributes a model's prediction to each input feature by considering all possible feature subsets
C)	Visualising decision boundaries of a classifier
D)	Selecting the best model from an ensemble
Ans:	

SECTION 2 — ADVANCED CODING & IMPLEMENTATION

Questions 51 – 100 | Read the code / snippet and choose the correct answer

Q51.	**What does the following NumPy code compute? import numpy as np X = np.array([[1,2],[3,4],[5,6]]) result = np.mean(X, axis=0) print(result)**
A)	[2. 4.] — row-wise mean
B)	[3. 4.] — column-wise mean, result is [3. 4.]
C)	[1.5 3.5 5.5] — element-wise
D)	[21.] — total sum
Ans:	

Q52.	**What is printed by the following? import numpy as np A = np.array([1,2,3,4]) B = A.reshape(2,2) B[0,0] = 99 print(A[0])**
A)	1 (A is unaffected)
B)	99 (reshape returns a view, so A shares memory)
C)	0
D)	Error: cannot assign to reshaped array
Ans:	

Q53.	**What does np.linalg.svd(X) return? X = np.random.randn(100, 10)**
A)	Only the singular values as a 1-D array
B)	Three arrays U, S, Vt — left singular vectors, singular values, right singular vectors — used in dimensionality reduction
C)	The eigenvalues and eigenvectors of X
D)	The inverse of the matrix X
Ans:	

Q54.	**What does the following code compute? import numpy as np X = np.array([[1,0],[0,1],[1,1]]) y = np.array([1,-1,1]) weights = np.linalg.lstsq(X, y, rcond=None)[0] print(weights)**
A)	The maximum likelihood weights
B)	The ordinary least squares solution to the linear system X @ w = y
C)	The gradient of the loss
D)	The eigenvalues of X
Ans:	

Q55.	What does the following broadcasting code output? import numpy as np a = np.array([[1],[2],[3]]) b = np.array([10,20,30]) print((a + b).shape)
A)	(3,)
B)	(3,3)
C)	(9,)
D)	ValueError: shapes not aligned
Ans:	

Q56.	What does the following Pandas operation return? import pandas as pd df = pd.DataFrame({'A':[1,2,None],'B':[4,None,6]}) print(df.isnull().sum())
A)	The total number of null values in the DataFrame
B)	A Series with the count of null values per column
C)	A boolean DataFrame
D)	The mean of non-null values per column
Ans:	

Q57.	What is the output of this Pandas code? import pandas as pd df = pd.DataFrame({'score':[80,90,70,85]}) result = df['score'].apply(lambda x: 'pass' if x >= 75 else 'fail') print(result.value_counts()['pass'])
A)	1
B)	2
C)	3
D)	4
Ans:	

Q58.	What does df.groupby('class')['score'].agg(['mean','std']) return?
A)	A single mean and std for the entire DataFrame
B)	A DataFrame indexed by 'class' with columns for mean and std of 'score' per class group
C)	A list of tuples
D)	An error because agg cannot take a list
Ans:	

Q59.	What is the purpose of pd.get_dummies(df, drop_first=True)?

A)	Drops all categorical columns from the DataFrame
B)	One-hot encodes categorical columns and drops the first dummy to avoid multicollinearity
C)	Converts all columns to float dtype
D)	Fills missing values with dummy variables
Ans:	

Q60.	**What does the following code produce? import pandas as pd df1 = pd.DataFrame({'key':['A','B'],'val1':[1,2]}) df2 = pd.DataFrame({'key':['B','C'],'val2':[3,4]}) result = pd.merge(df1, df2, on='key', how='outer') print(len(result))**
A)	1
B)	2
C)	3
D)	4
Ans:	

Q61.	**What does the following Scikit-learn code do? from sklearn.pipeline import Pipeline from sklearn.preprocessing import StandardScaler from sklearn.linear_model import LogisticRegression pipe = Pipeline([('scaler', StandardScaler()), ('clf', LogisticRegression())]) pipe.fit(X_train, y_train)**
A)	Trains only the scaler
B)	Chains preprocessing and model training so that fit applies scaling then trains the classifier on scaled data, preventing data leakage
C)	Trains the classifier without scaling
D)	Performs cross-validation automatically
Ans:	

Q62.	**What is the output type of cross_val_score(estimator, X, y, cv=5, scoring='accuracy')?**
A)	A single float representing average accuracy
B)	A NumPy array of 5 accuracy scores, one per fold
C)	A dict with train and test scores
D)	A confusion matrix
Ans:	

Q63.	**What does the following Grid Search code select? from sklearn.model_selection import GridSearchCV from sklearn.svm import SVC param_grid = {'C': [0.1, 1, 10], 'kernel': ['linear','rbf']} gs =**

	GridSearchCV(SVC(), param_grid, cv=5) gs.fit(X_train, y_train) print(gs.best_params_)
A)	The parameter values with the highest training accuracy
B)	The hyperparameter combination with the best mean cross-validation score
C)	The last combination tried in the grid
D)	The parameters that minimise overfitting on all folds
Ans:	

Q64.	What does ColumnTransformer in Scikit-learn allow you to do?
A)	Apply different transformations to different subsets of columns in a single step
B)	Transform columns sequentially one after another
C)	Automatically select the best transformation for each column
D)	Merge multiple DataFrames into one
Ans:	

Q65.	What does the following code output? from sklearn.linear_model import Ridge import numpy as np X = np.array([[1,1],[1,2],[2,2]]) y = np.array([1,2,3]) model = Ridge(alpha=0.0) model.fit(X,y) print(round(model.score(X,y), 2))
A)	0.0 — Ridge with alpha=0 always underfits
B)	1.0 — with alpha=0 Ridge reduces to OLS which perfectly fits 3 points with 2 features
C)	0.5 — average prediction
D)	Error: singular matrix
Ans:	

Q66.	What does the following manual logistic regression update rule implement? w += lr * X.T @ (y - sigmoid(X @ w))
A)	Stochastic Gradient Descent step
B)	Gradient ascent on the log-likelihood (equivalent to gradient descent on the cross-entropy loss)
C)	Newton-Raphson update
D)	L2 regularised weight update
Ans:	

Q67.	What is the output of this decision tree code? from sklearn.tree import DecisionTreeClassifier clf =

	DecisionTreeClassifier(max_depth=None) clf.fit(X_train, y_train) print(clf.score(X_train, y_train))
A)	Approximately 0.5 for a balanced dataset
B)	1.0 — an unconstrained decision tree memorises training data perfectly
C)	The validation accuracy
D)	Depends on the random seed only
Ans:	

Q68.	**What does the following code compute in a k-Means implementation? centroids = np.array([X[labels == k].mean(axis=0) for k in range(K)])**
A)	Assigns each point to the nearest centroid
B)	Updates centroids to the mean of all points assigned to each cluster
C)	Initialises centroids randomly
D)	Computes the within-cluster variance
Ans:	

Q69.	**What does this PCA code achieve? from sklearn.decomposition import PCA pca = PCA(n_components=0.95) X_reduced = pca.fit_transform(X_scaled)**
A)	Reduces to exactly 2 components
B)	Selects the minimum number of principal components that explain 95% of the total variance
C)	Normalises the data to unit variance
D)	Computes the covariance matrix of X_scaled
Ans:	

Q70.	**What does the following code evaluate? from sklearn.metrics import roc_auc_score y_pred_proba = model.predict_proba(X_test)[:,1] print(roc_auc_score(y_test, y_pred_proba))**
A)	The accuracy at a 0.5 threshold
B)	The area under the ROC curve, measuring discriminative ability across all classification thresholds
C)	The F1-Score at threshold 0.5
D)	The log-loss of the model
Ans:	

Q71.	**What does the following Keras layer do? tf.keras.layers.Dense(128, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.01))**

A)	A fully connected layer with 128 neurons, ReLU activation, and L2 weight regularisation with lambda=0.01
B)	A convolutional layer with 128 filters
C)	A dropout layer with 12.8% drop rate
D)	A batch normalisation layer with 128 features
Ans:	

Q72.	**What does model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']) assume about the target labels?**
A)	Labels are one-hot encoded vectors
B)	Labels are integers representing class indices (not one-hot encoded)
C)	Labels are continuous float values
D)	Labels are probability distributions
Ans:	

Q73.	**What is the output shape of the following Conv2D operation? # Input: (batch=32, H=28, W=28, C=1) tf.keras.layers.Conv2D(32, kernel_size=3, strides=1, padding='same')**
A)	(32, 26, 26, 32)
B)	(32, 28, 28, 32) — 'same' padding preserves spatial dimensions
C)	(32, 28, 28, 1)
D)	(32, 32, 32, 32)
Ans:	

Q74.	**What does tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True) do during training?**
A)	Stops training after exactly 5 epochs
B)	Stops training if validation loss does not improve for 5 consecutive epochs and restores the model weights from the best epoch
C)	Reduces the learning rate after 5 epochs
D)	Evaluates the model every 5 epochs
Ans:	

Q75.	**In the following LSTM code, what does return_sequences=True control? tf.keras.layers.LSTM(64, return_sequences=True)**
A)	Returns only the final hidden state of shape (batch, 64)
B)	Returns the hidden state at every timestep, output shape (batch, timesteps, 64), required when stacking LSTM layers

C)	Enables bidirectional processing
D)	Returns the cell state instead of the hidden state
Ans:	

Q76.	**What does the following TF-IDF code compute? from sklearn.feature_extraction.text import TfidfVectorizer vec = TfidfVectorizer(max_features=1000) X = vec.fit_transform(corpus)**
A)	A bag-of-words count matrix
B)	A sparse matrix of TF-IDF scores where common words across documents are down-weighted
C)	Word embeddings for each term
D)	Character-level n-gram frequencies
Ans:	

Q77.	**What is the shape of the attention score matrix in a single-head self-attention layer for a sequence of length T?**
A)	(T, d_model)
B)	(T, T) — each token attends to every other token
C)	(d_model, d_model)
D)	(batch, T)
Ans:	

Q78.	**What does the following HuggingFace code accomplish? from transformers import pipeline classifier = pipeline('sentiment-analysis') result = classifier('Machine learning is fascinating!') print(result)**
A)	Fine-tunes a BERT model on custom data
B)	Runs inference using a pre-trained sentiment analysis model and returns label and confidence score
C)	Tokenises the input text only
D)	Trains a new transformer from scratch
Ans:	

Q79.	**What is the purpose of tokenizer.encode_plus(..., padding='max_length', truncation=True, max_length=128) in BERT?**
A)	Converts tokens to word2vec embeddings
B)	Tokenises text, pads shorter sequences and truncates longer ones to exactly 128 tokens, returning input_ids and attention_mask
C)	Performs stemming and lemmatisation

D)	Generates BERT embeddings directly
Ans:	

Q80.	**What does cosine_similarity between two word embedding vectors measure? from sklearn.metrics.pairwise import cosine_similarity sim = cosine_similarity(vec1.reshape(1,-1), vec2.reshape(1,-1))**
A)	The Euclidean distance between the two vectors
B)	The cosine of the angle between the vectors, ranging from -1 to 1, measuring semantic similarity regardless of vector magnitude
C)	The dot product normalised by vector length squared
D)	The probability that the two words co-occur
Ans:	

Q81.	**What does the following XGBoost snippet control? xgb.XGBClassifier(n_estimators=300, max_depth=4, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0)**
A)	Only the tree depth and number of trees
B)	300 trees of depth 4, slow learning rate (shrinkage), row and column subsampling for regularisation, plus L2 weight regularisation
C)	A random forest with 300 trees
D)	A neural network with 300 layers
Ans:	

Q82.	**What does the following stacking ensemble code achieve? from sklearn.ensemble import StackingClassifier stacking = StackingClassifier(estimators=[('rf', rf), ('svm', svm)], final_estimator=LogisticRegression()) stacking.fit(X_train, y_train)**
A)	Trains only the best base model
B)	Trains RF and SVM as base models and uses their out-of-fold predictions as input to train a Logistic Regression meta-learner
C)	Averages predictions from RF and SVM
D)	Trains RF and SVM sequentially on residuals
Ans:	

Q83.	**What does the following SHAP code visualise? import shap explainer = shap.TreeExplainer(model) shap_values = explainer.shap_values(X_test) shap.summary_plot(shap_values, X_test)**
A)	The ROC curve of the model
B)	The contribution (SHAP value) of each feature to predictions across all test samples, showing global feature importance and direction of effect

C)	The confusion matrix
D)	The learning curves of the model
Ans:	

Q84.	**In the following code, what does StratifiedKFold ensure? from sklearn.model_selection import StratifiedKFold skf = StratifiedKFold(n_splits=5) for train_idx, val_idx in skf.split(X, y):**
A)	Each fold has the same number of samples regardless of class
B)	Each fold preserves the same class proportion as the full dataset, crucial for imbalanced classification
C)	Folds are split by time order
D)	Each fold contains only one class
Ans:	

Q85.	**What does the following SMOTE code address? from imblearn.over_sampling import SMOTE X_res, y_res = SMOTE(random_state=42).fit_resample(X_train, y_train)**
A)	Feature normalisation for SVM
B)	Class imbalance — SMOTE synthesises new minority class samples by interpolating between existing ones
C)	Dimensionality reduction
D)	Outlier removal from the training set
Ans:	

Q86.	**What does the following code compute for model evaluation? from sklearn.calibration import calibration_curve fraction_pos, mean_pred = calibration_curve(y_test, y_prob, n_bins=10)**
A)	The precision-recall curve
B)	Compares predicted probabilities against actual positive fractions per bin to assess probability calibration quality
C)	The ROC curve in a different format
D)	The distribution of predicted probabilities
Ans:	

Q87.	**What does the following Q-learning update implement? Q[s,a] = Q[s,a] + alpha*(reward + gamma*np.max(Q[s_next,:]) - Q[s,a])**
A)	Policy gradient update
B)	The Bellman equation update for Q-values: current estimate corrected by the TD error (reward + discounted future value - current Q)

C)	Actor-critic update
D)	Monte Carlo return estimate
Ans:	

Q88.	**What does the following code implement in a neural network?** `def positional_encoding(seq_len, d_model):  PE = np.zeros((seq_len, d_model))  for pos in range(seq_len):  for i in range(0, d_model, 2):  PE[pos,i] = np.sin(pos/10000**(i/d_model))  PE[pos,i+1] = np.cos(pos/10000**(i/d_model))  return PE`
A)	Word embedding lookup
B)	Sinusoidal positional encoding for Transformers, injecting position information into token representations
C)	Attention score computation
D)	Layer normalisation
Ans:	

Q89.	**What is the purpose of the following gradient clipping call in PyTorch?** `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)`
A)	Setting all gradients to 1.0
B)	Scaling gradients so their global norm does not exceed 1.0, preventing exploding gradients in RNN/Transformer training
C)	Freezing model parameters
D)	Normalising model weights directly
Ans:	

Q90.	**What does the following VAE code component represent?** `z = mu + eps * torch.exp(0.5 * log_var) # where eps = torch.randn_like(log_var)`
A)	Standard autoencoder bottleneck
B)	The reparameterisation trick enabling backpropagation through a stochastic sampling step in a Variational Autoencoder
C)	Dropout in the latent space
D)	The KL divergence computation
Ans:	

Q91.	**What does the following code implement?** `def batch_norm(X, gamma, beta, eps=1e-5):  mu = X.mean(axis=0)  var = X.var(axis=0)  X_norm = (X - mu) / np.sqrt(var + eps)  return gamma * X_norm + beta`
A)	Layer Normalisation

B)	Batch Normalisation: normalises features across the batch dimension then applies learnable scale (gamma) and shift (beta)
C)	Instance Normalisation
D)	Dropout regularisation
Ans:	

Q92.	**What does the following multi-head attention code compute? scores = (Q @ K.transpose(-2,-1)) / math.sqrt(d_k) weights = F.softmax(scores, dim=-1) output = weights @ V**
A)	Positional encoding for the transformer
B)	Scaled dot-product attention: computes attention weights from query-key dot products scaled by sqrt(d_k) and uses them to aggregate value vectors
C)	Layer normalisation of Q,K,V
D)	Feed-forward sub-layer computation
Ans:	

Q93.	**What does the following code demonstrate about PyTorch autograd? import torch x = torch.tensor(3.0, requires_grad=True) y = x**3 + 2*x y.backward() print(x.grad)**
A)	0.0
B)	9.0 — PyTorch computes dy/dx = 3x^2 + 2 at x=3 → 27+2=29... wait, 3*(9)+2=29
C)	29.0 — dy/dx = 3x^2 + 2 = 3(9)+2 = 29
D)	3.0
Ans:	

Q94.	**What does the following learning rate scheduler do? scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)**
A)	Reduces LR by half every 50 epochs
B)	Anneals the learning rate following a cosine curve from the initial LR to near zero over T_max=50 epochs, smoothly reducing it
C)	Increases LR for the first 50 steps then keeps it constant
D)	Applies warm restarts every epoch
Ans:	

Q95.	**What is the purpose of torch.no_grad() in the following inference code? with torch.no_grad(): outputs = model(X_test) preds = outputs.argmax(dim=1)**
A)	Disables model layers from processing input

B)	Disables gradient computation and tracking to save memory and speed up inference when gradients are not needed
C)	Freezes model weights permanently
D)	Runs the model in half-precision mode
Ans:	

Q96.	**What does the following data augmentation pipeline achieve? transforms.Compose([transforms.RandomHorizontalFlip(p=0.5), transforms.RandomCrop(32, padding=4), transforms.ColorJitter(brightness=0.2), transforms.ToTensor(), transforms.Normalize(mean, std)])**
A)	Preprocesses test images only
B)	Applies random flips, crops, and colour jitter during training to artificially increase dataset diversity and reduce overfitting, followed by normalisation
C)	Converts images to grayscale
D)	Reduces image resolution to 32x32
Ans:	

Q97.	**What does the following code compute and what is it used for? from sklearn.inspection import permutation_importance result = permutation_importance(model, X_test, y_test, n_repeats=10) print(result.importances_mean)**
A)	The gradient of the loss with respect to each feature
B)	Feature importance by measuring the mean decrease in model performance when each feature's values are randomly permuted, applicable to any model
C)	Shapley values for the test set
D)	The correlation between each feature and the target
Ans:	

Q98.	**What does the following code implement in the context of hyperparameter optimisation? from sklearn.model_selection import RandomizedSearchCV from scipy.stats import randint, uniform param_dist = {'n_estimators': randint(50,500), 'max_depth': randint(3,15), 'learning_rate': uniform(0.01,0.3)} rs = RandomizedSearchCV(model, param_dist, n_iter=50, cv=5) rs.fit(X_train, y_train)**
A)	Exhaustive grid search over all combinations
B)	Random sampling of 50 hyperparameter combinations from specified distributions, then 5-fold CV evaluation — more efficient than grid search in high dimensions
C)	Bayesian optimisation of hyperparameters

D)	Automated neural architecture search
Ans:	

Q99.	**What does the following t-SNE code visualise and what are its limitations? `from sklearn.manifold import TSNE X_2d = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(X_scaled)`**
A)	A linear projection like PCA; limitations include loss of variance information
B)	A non-linear dimensionality reduction for visualisation that preserves local structure; limitations include non-determinism, no out-of-sample projection, and axes are not interpretable
C)	A clustering algorithm output; limitations include fixed number of clusters
D)	A feature importance map; limitations include computational cost only
Ans:	

— End of Machine Learning Mastery Quiz —

Write your answer (A / B / C / D) in each 'Ans:' box. Total: 100 Marks